# MINIMALISMUS

Rekursion — Scheme

Lambda-Kalkül

Typentheorie — Pie

## I REKURSION

### 1 Lisp und Scheme

| | | |
|---|---|---|
| FORTRAN | (1957) | IBM |
| LISP | (1958) | MIT |
| ↳ Scheme | (1970er) | MIT |

Orthogonal

REPL : Read - Eval - Print - Loop

### 2 Zahlen, Symbole, Listen

ATOME { Zahlen, Symbole

| | | |
|---|---|---|
| Symbole | "HAW" | "HAW" |
| | HAW | |

Listen

### 3 EVALUATION

Evaluationsregeln :

(E1) Ausdruck / Term : syntaktisch korrekte Zeichenkette

| | | | |
|---|---|---|---|
| 42 | 'haw | '(+ 40 2) | (+ 40 2) |

(E2) Jeder Ausdruck wird evaluiert

Rückgabe: Wert

(E3) Zahlen sind selbst-evaluierend (Literale)

(E4) An den zusammengesetzten Zeichenketten werden als Name interpretiert

(E5) Ist der zu evaluierende Ausdruck eine Liste, dann muss das erste Element eine Funktion sein. Die anderen Elemente werden ausgewertet und an diese Funktion übergeben.

Aufg.:

| | |
|---|---|
| $f(1,x)$ | (f 1 x) |
| $f(g(2,3),x)$ | (f (g 2 3) x) |
| $f(g(2,3),h(4,4,2))$ | (f (g 2 3) (h 4 4 2)) |
| $f(g(2,3),h(4,g(5,6),2))$ | (f (g 2 3) (h 4 (g 5 6) 2)) |

(E6) Es gibt Spezialoperatoren, bei denen nicht alle Argumente evaluiert werden.

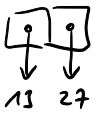
Aufg.:
① Liste '(H A U) soll Name HAU behommen.
② Ich möchte statt (+ 40 2) in Zukunft (PLUS 40 2) schreiben können.

## 4 S-Ausdrücke

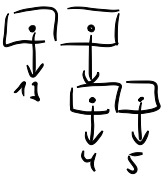
Erinnerung: Paar (13, 27)

(cons 13 27)

car cdr




(cons 13 (cons 4 5))

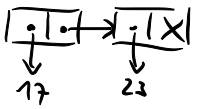



'() ⊠

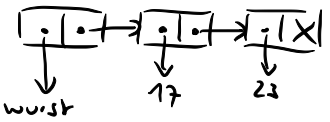
(cons 23 '())




(cons 17 (cons 23 '()))

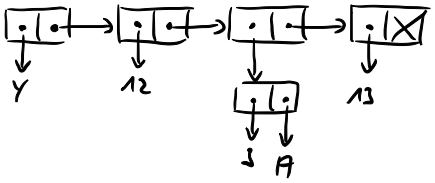



(cons 'wurst (cons 17 (cons 23 '())))

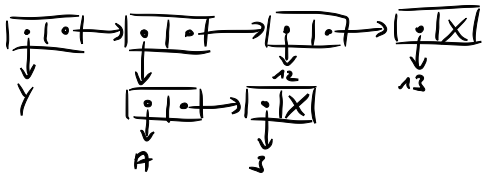



Aufg.: Grafisch darstellen

- (list 'Y 12 (cons 3 'A) 13)




- (list 'Y Clist 'A 3) 12 13)




Def.: Atome und die leere Liste sind S-Ausdrücke.
Wenn X und Y S-Ausdrücke sind,
dann ist auch (cons X Y) ein S-Ausdruck.

Bsp.: 5 wurst () 23
(cons 5 'wurst) (5 . wurst)
(cons 23 (cons 5 'wurst))

Def.: Die leere Liste () ist eine Liste.
Ist S irgendein S-Ausdruck
und L eine Liste, dann ist
(cons S L) auch eine Liste.

## 5. Funktionen

Funktionen sind First-Class-Objekte

(lambda (x) (+ x 1))

**Aufg:** ① Funktion für $x \mapsto 5x^2$

② Funktion für $n \mapsto \sum_{k=1}^{n} k = \frac{n(n+1)}{2}$

① (lambda (x) (* 5 x x))

② (lambda (n) (quotient (* n (+ n 1)) 2))

**Details zum Ablauf** für (add1 (* 3 4))

(define add1 (lambda (x) (+ x 1)))

① ((lambda (x) (+ x 1)) (* 3 4))

② ((lambda (x) (+ x 1)) 12)

Variablen-Shadowing →

(define x 12)

evaluiere (+ x 1)

((x . 42) (y. 12) ...)

((x. 12) (x. 42) (y. 12) ...)

(define x 41)

(+ x 1) ---> ? 42

((lambda (x) (+ x 1)) 99) ---> ? 100

(+ x 1) ---> ? 42